



INTRODUCTION TO GIT & GITHUB

Git, GitHub



git





TỔNG QUAN 5 BUỔI HỌC

Lộ trình cho học PNV28

Buổi 1: Giới thiệu Git, Khái niệm cơ bản, Init / Add / Commit / Log

Làm quen Git, cài đặt, tạo account GitHub, hiểu 3 vùng dữ liệu & lệnh Terminal đầu tiên.

Buổi 2: Git Branching, Merge & Conflict Resolution

Tạo nhánh độc lập, gộp code và kỹ thuật tự tay xử lý đụng độ code thủ công.

Buổi 3: GitHub Remote - Push / Pull / Clone & Pull Requests

Đưa dự án lên đám mây, đồng bộ dữ liệu và quy trình duyệt code (PR) chuẩn team.

Buổi 4: Workflow Thực Tế (Git Flow, Feature Branch) & Teamwork

Mô hình Git Flow chuẩn doanh nghiệp, làm việc nhóm an toàn với GitHub.

Buổi 5: Quản Lý Dự Án Jira & Liên Kết GitHub

Tạo Project/Board trên Jira, theo dõi công việc và tự động hóa trạng thái với Git.

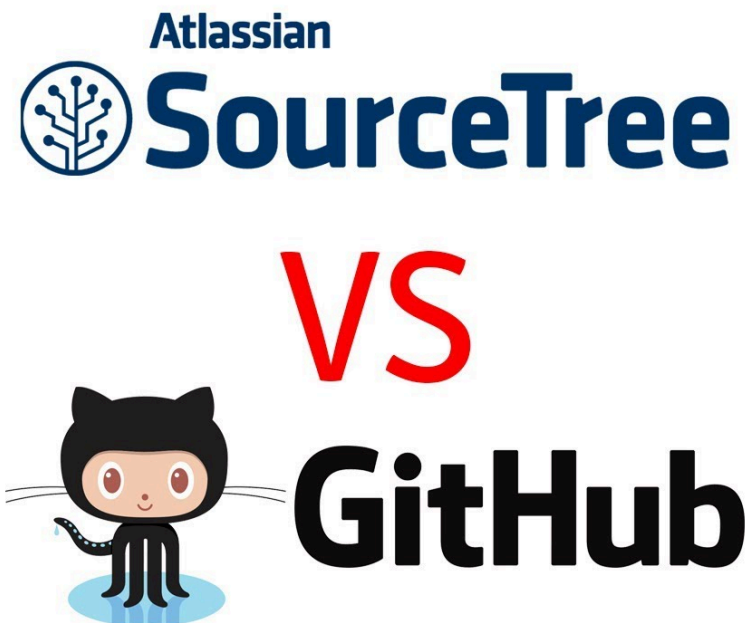


BUỔI 1: OVERVIEW NỘI DUNG HỌC

Cài Đặt, Khái Niệm Cơ Bản & Câu Lệnh Terminal Khởi Đầu

Các Nội Dung Chính

1. Hiểu nhanh Git là gì & phân biệt Git vs GitHub / SourceTree.
2. Cài đặt Git & Đăng ký tài khoản GitHub.
3. Nắm vững 3 Vùng Dữ Liệu Trong Git.
4. Các câu lệnh Terminal căn bản: `init`, `status`, `add`, `commit`, `log`.





Buổi 1: Git Là Gì & Tại Sao Lại Dùng Git?

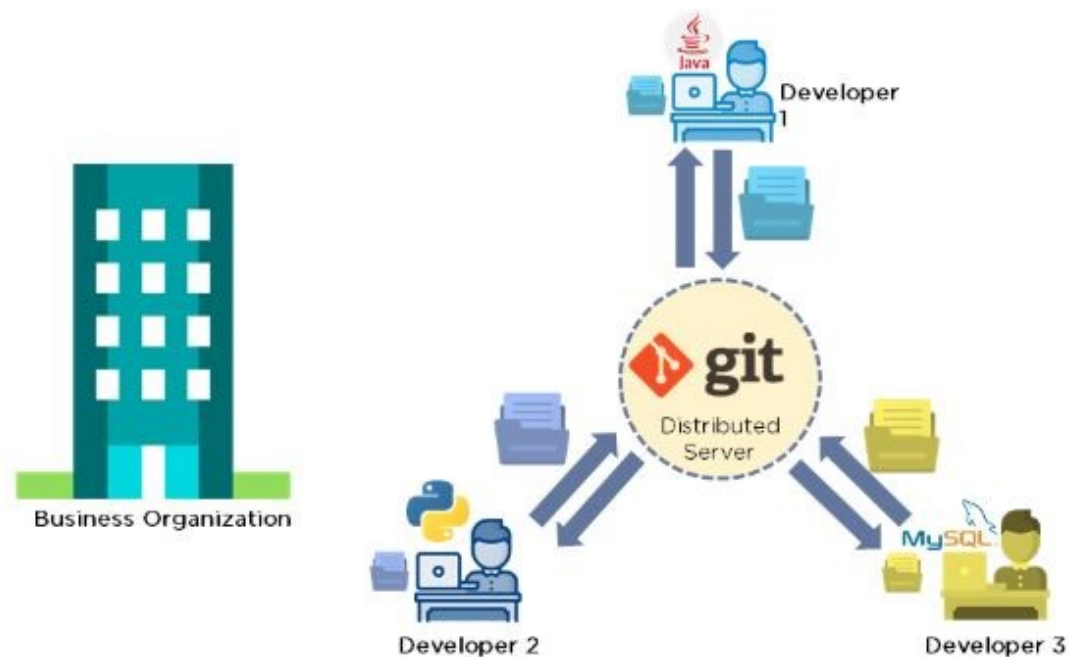
Giới thiệu cực kỳ dễ hiểu dành cho người mới

Git Là Gì? (Nhật Ký Lưu Bản Vẽ)

Giống như nút **"Save / Undo"** thông minh. Giúp bạn lưu lại từng mốc thời gian của file code để quay lại bất kỳ lúc nào nếu lỡ làm hỏng.

Mục Đích Sử Dụng

- Tránh việc phải lưu file thủ công: `Code_v1.zip`, `Code_Final.zip`.
- Cho phép 10 người cùng sửa 1 dự án mà không lo đề mất code của nhau.





Buổi 1: Phân Biệt Git, GitHub & SourceTree

Đừng nhầm lẫn giữa engine, server đám mây và giao diện đồ họa!

1. Git (Động cơ)

Phần mềm chạy dòng lệnh cài trên máy bạn để lưu lịch sử code local.

2. GitHub / Bitbucket

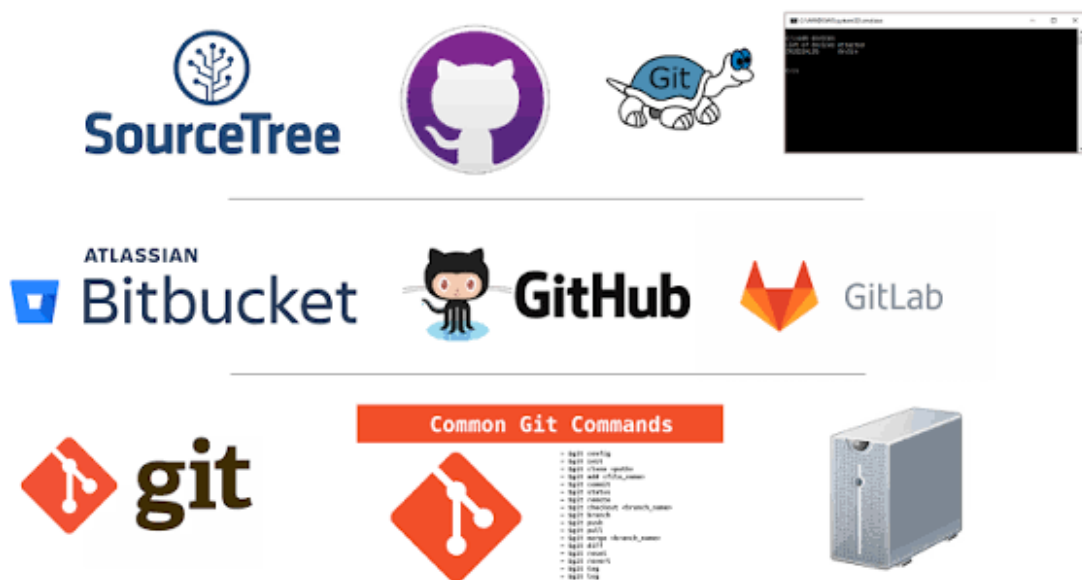
Website lưu trữ kho chứa code lên đám mây để chia sẻ cho cả team.

3. SourceTree / Client

Phần mềm giao diện đồ họa (nút bấm) giúp thao tác Git không cần gõ lệnh.

Mini Practice 1.1

Câu hỏi nhanh: Bạn A dùng VS Code sửa file, bấm nút *Commit* rồi đẩy lên *GitHub*. Trong luồng này, đâu là Git và đâu là GitHub?





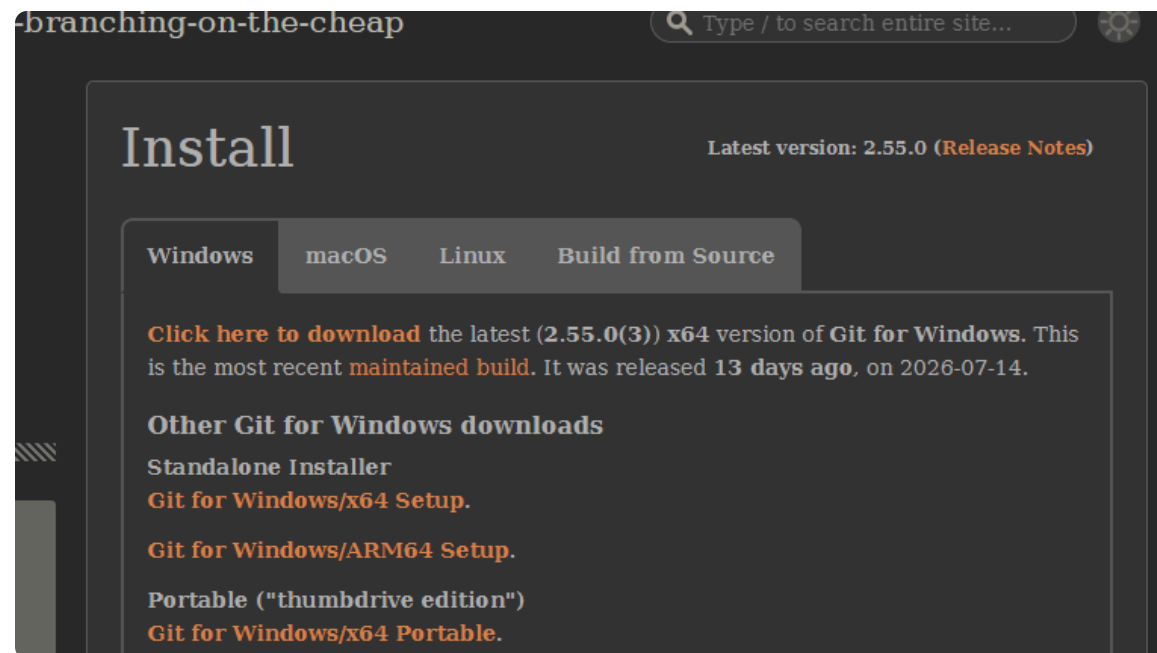
Buổi 1: Cài Đặt Git Cho Windows & Mac

Hướng dẫn từng bước tải và cài đặt phần mềm

Các Bước Thực Hiện:

1. Tải bộ cài: Truy cập <https://git-scm.com/downloads> chọn installer phù hợp.
2. Cài đặt: Giữ nguyên cài đặt mặc định (Default). Tích chọn Git Bash trên Windows.
3. Kiểm tra: Mở Terminal / Git Bash gõ lệnh test:

```
git --version
```





Buổi 1: Thiết Lập Identity & Account GitHub

Khởi tạo thông tin người dùng local và tài khoản web

1. Thao Tác Terminal (Config Email/Name)

```
git config --global user.name "Nguyen Van A"  
git config --global user.email "a.nguyen@gmail.com"  
git config --list
```

Email config trùng 100% với Email đăng ký GitHub!

2. Đăng Ký GitHub Web

Truy cập <https://github.com> → bấm **Sign Up** → Điền Email, Pass, Username → Xác minh OTP qua mail.

Mini Practice 1.2: Thực Hành Cài Đặt & Cấu Hình

1. Chạy lệnh `git config --list` trên máy cá nhân và chụp lại màn hình tên/email đã đặt.
2. Đăng ký thành công 1 tài khoản GitHub cá nhân.



Buổi 1: 3 Vùng Dữ Liệu Trong Git

Cực kỳ dễ hiểu qua mô hình Đi Siêu Thị

1. Working Directory

Nơi viết code trực tiếp trên VS Code (Giống hàng hóa nằm trên kệ siêu thị).

2. Staging Area (Khu vực chờ)

Dùng lệnh `git add` để chọn file (Giống nhặt đồ bỏ vào Giỏ Hàng).

3. Local Repository (Kho lưu trữ)

Dùng `git commit` để đóng gói vĩnh viễn (Giống tính tiền nhận Hóa Đơn).





Buổi 1: Thao Tác Dòng Lệnh Git Căn Bản

Các lệnh làm việc local chạy hàng ngày

`git init` : Biến 1 thư mục thường thành kho chứa Git.

`git status` : Kiểm tra trạng thái các file (đã sửa / chưa add).

`git add .` : Đưa file vào Staging Area (Giỏ hàng).

`git commit -m "Mô tả"` : Chốt phiên bản kèm lời nhắn.

`git log --oneline` : Xem lại lịch sử các commit.



Mini Practice 1.3: Bài Tập Commit Đầu Tiên

Tạo thư mục `my-project` → Chạy `git init` → Tạo file `index.html` → Chạy `git add .` → Commit với tin nhắn "*Init project*" → Kiểm tra lại bằng `git log`.

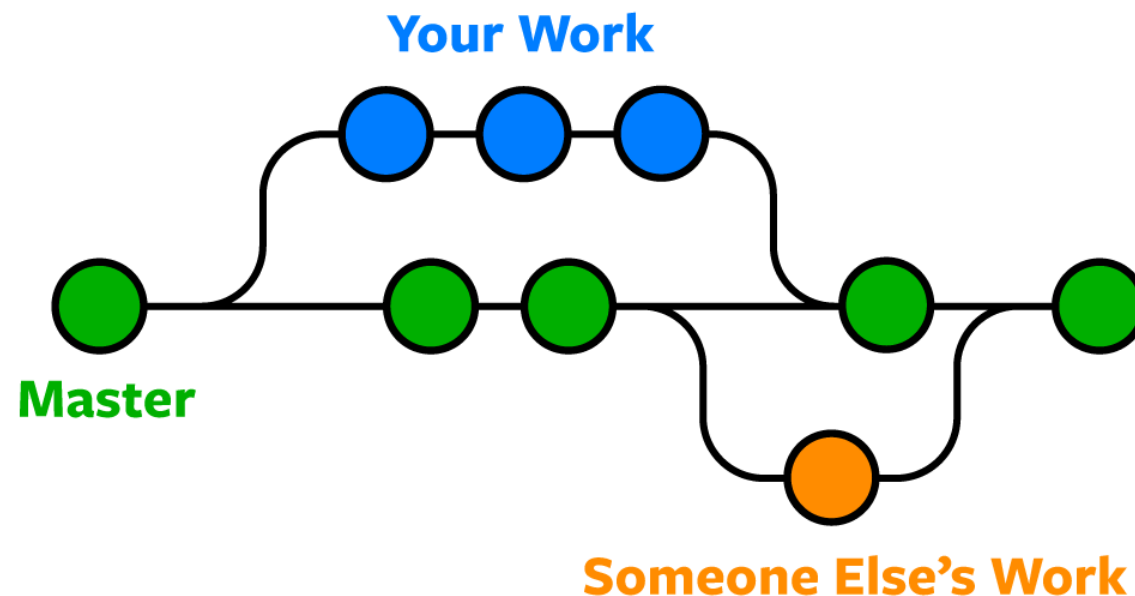


BUỔI 2: OVERVIEW NỘI DUNG HỌC

Git Branching, Merge & Conflict Resolution

Các Nội Dung Chính

1. Khái niệm nhánh (Branch) và tại sao phải chia nhánh?
2. Các lệnh thao tác nhánh: `branch`, `checkout`, `checkout -b`.
3. Gộp nhánh (Merge code) về nhánh chính `main`.
4. Nhận diện và tự tay giải quyết Merge Conflict thủ công.



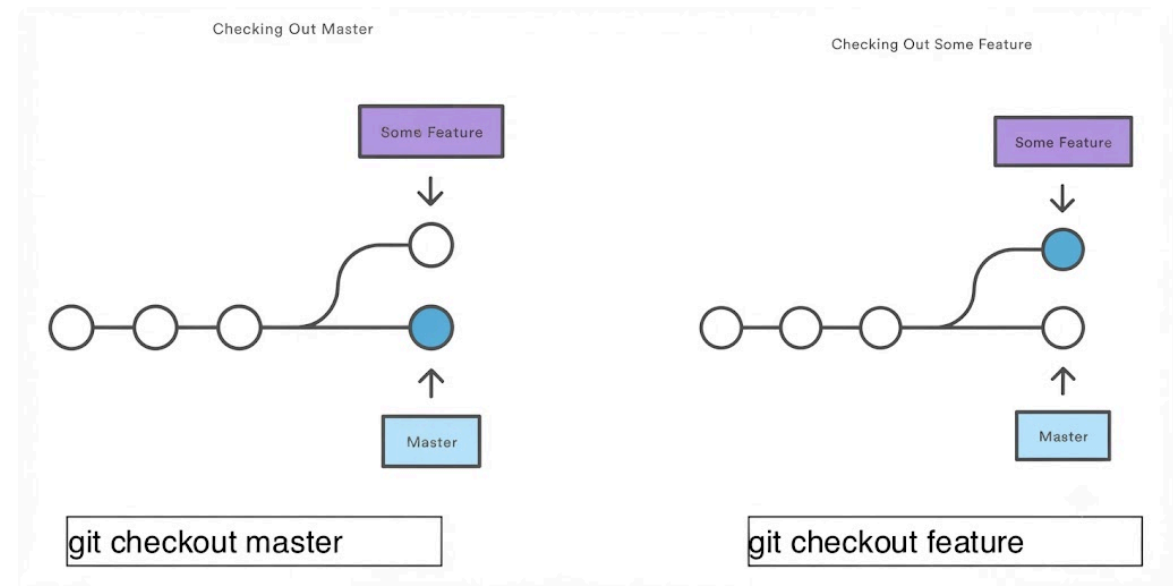


Buổi 2: Kỹ Thuật Chia Nhánh (Branching)

Giúp bạn làm tính năng mới cực kỳ an toàn

Lệnh Thao Tác Nhánh:

- Xem danh sách nhánh: `git branch`
- Tạo nhánh mới: `git branch feature/login`
- Chuyển sang nhánh mới: `git checkout feature/login`
- Tạo & chuyển nhánh: `git checkout -b feature/login`



Mini Practice 2.1

Tại dự án `my-project`, tạo nhánh mới tên `feature/header`, chuyển sang nhánh đó, tạo file `header.html` rồi thực hiện commit.



Buổi 2: Quy Trình Merge & Fix Conflict

Cách xử lý đụng độ code khi làm việc chung file

1. Gộp Nhánh (Merge)

```
git checkout main
git merge feature/login
```

2. 4 Bước Fix Conflict Thủ Công

B1: Mở file lỗi trên VS Code tìm vạch <<<<<< HEAD.

B2: Thảo luận với bạn cùng team để chọn đoạn code đúng.

B3: Xóa vạch ký hiệu, chạy `git add .`

B4: Chốt commit: `git commit -m "Fix conflict"`

```
TS walkThroughPart.ts src/vs/workbench/parts/welcome/walkThrough/electron-browser
406 > > > > > > > snippet: i
407 > > > > > > > });
408 > > > > > > > }));
409 > > > > > > > });
    Accept Current Change | Accept Incoming Change | Accept Both Changes | Compare Changes
410 <<<<<< HEAD (Current Change)
411 > > > > > > > this.updateSizeClasses();
412 > > > > > > > this.multiCursorModifier();
413 > > > > > > > this.contentDisposables.push(this.configurationService.onDidU
414 =====
415 > > > > > > > this.toggleSizeClasses();
416 >>>>>> Test (Incoming Change)
417 > > > > > > > if (input.onReady) {
418 > > > > > > >     input.onReady(innerContent);
419 > > > > > > > }
420 > > > > > > > this.scrollbar.scanDomNode();
421 > > > > > > > this.loadTextEditorViewState(input.getResource());
422 > > > > > > > this.updatedScrollPosition();
423 > > > > > > > });
424 > > > > > > > }
```

Mini Practice 2.2: Thử Tạo & Sửa Conflict

Tạo 2 nhánh cùng sửa dòng 1 file `README.md`. Tiến hành Merge nhánh 2 vào nhánh 1 để kích hoạt conflict, sau đó dùng VS Code sửa và commit lại!



BUỔI 3: OVERVIEW NỘI DUNG HỌC

GitHub Remote Repository & Quy Trình Pull Request (PR)

Các Nội Dung Chính

1. Kết nối Git Local với Remote Repo trên GitHub Web.
2. Thành thạo 3 lệnh đồng bộ: `clone`, `push`, `pull`.
3. Quy trình gửi Pull Request (PR) để xin gộp code.
4. Cách phân công Assignee, chỉ định Reviewer và Comment duyệt code.





Buổi 3: Kết Nối GitHub Remote (Clone/Push/Pull)

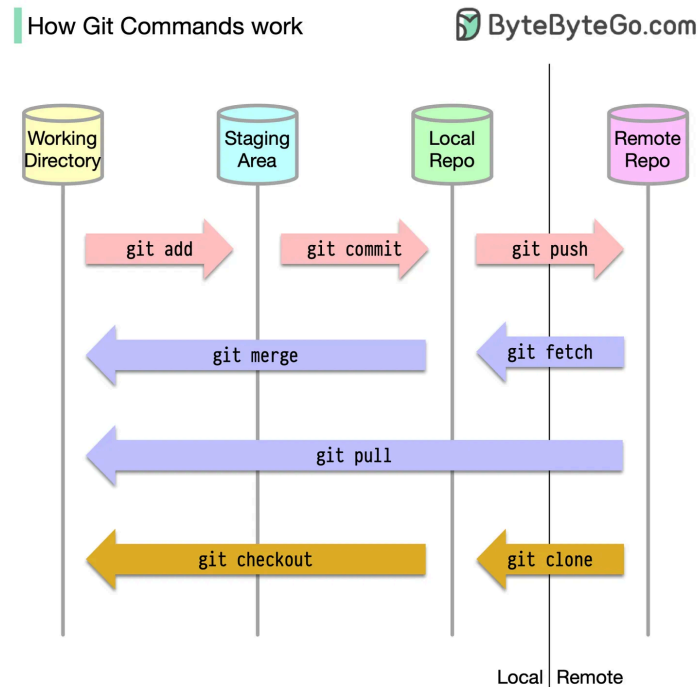
Đưa project từ máy cá nhân lên server đám mây

1. Liên Kết Remote Repo

```
git remote add origin <URL-GitHub>
```

2. 3 Lệnh Thao Tác Chính

- `git clone <URL>` : Tải toàn bộ Repo từ GitHub về máy mới.
- `git push origin <branch>` : Đẩy commit từ máy lên GitHub.
- `git pull origin <branch>` : Tải code mới nhất từ GitHub về máy.



Mini Practice 3.1

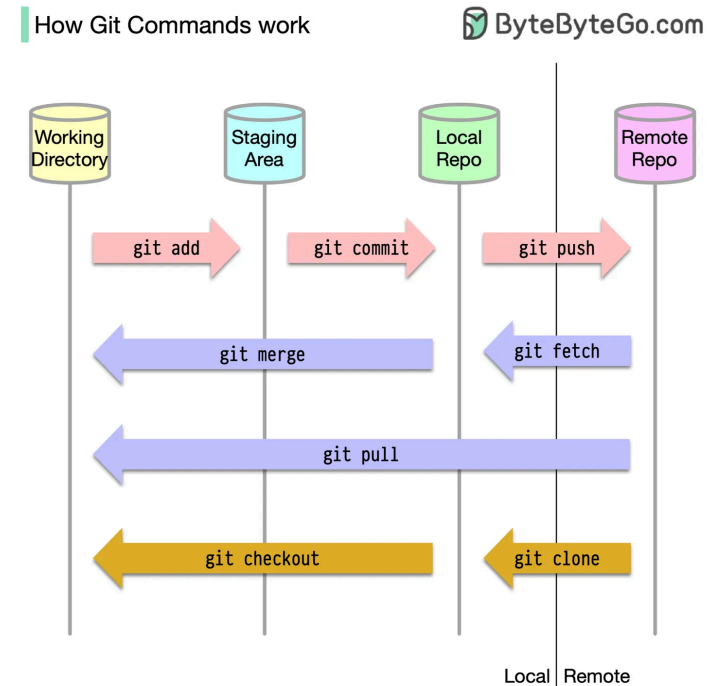
Tạo Repo mới trên GitHub web → Liên kết với project dưới máy qua `git remote add origin` → Push nhánh `main` lên GitHub.

Buổi 3: Quy Trình Mở & Duyệt Pull Request (PR)

Quy chuẩn kiểm tra chất lượng code trước khi gộp nhánh main

Các Bước Thực Hiện PR Chuyên Nghiệp:

1. Push nhánh: `git push origin feature/user-profile`
2. Mở PR: Vào GitHub bấm **Compare & pull request**.
3. Description: Nhập mô tả công việc đã hoàn thành.
4. Gán người: Chọn bản thân ở **Assignees**, chọn Leader ở **Reviewers**.
5. Review & Merge: Leader xem Diff, Comment, Approve → Bấm **Merge Pull Request**.



Mini Practice 3.2: Bài Tập Làm Việc Cặp

Học viên A tạo nhánh feature, push lên GitHub và mở PR → Gán Học viên B làm Reviewer → Học viên B vào comment check code và bấm Approve → Tiến hành Merge PR.

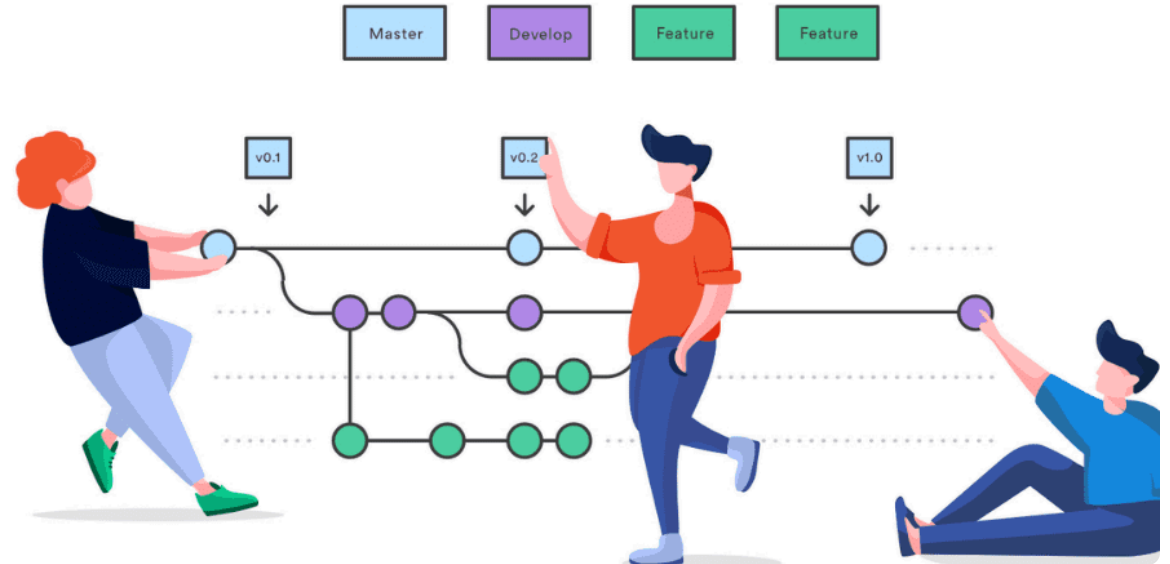


BUỔI 4: OVERVIEW NỘI DUNG HỌC

Workflow Thực Tế (Git Flow) & Làm Việc Nhóm

Các Nội Dung Chính

1. Tại sao dự án thực tế cần mô hình chuẩn (Workflow)?
2. Chi tiết Mô hình nhánh Git Flow (Master, Develop, Feature, Hotfix).
3. Quy tắc phối hợp làm việc nhóm an toàn trên GitHub.
4. Các VS Code Extension hỗ trợ dùng Git tốt hơn (GitLens, Git Graph).

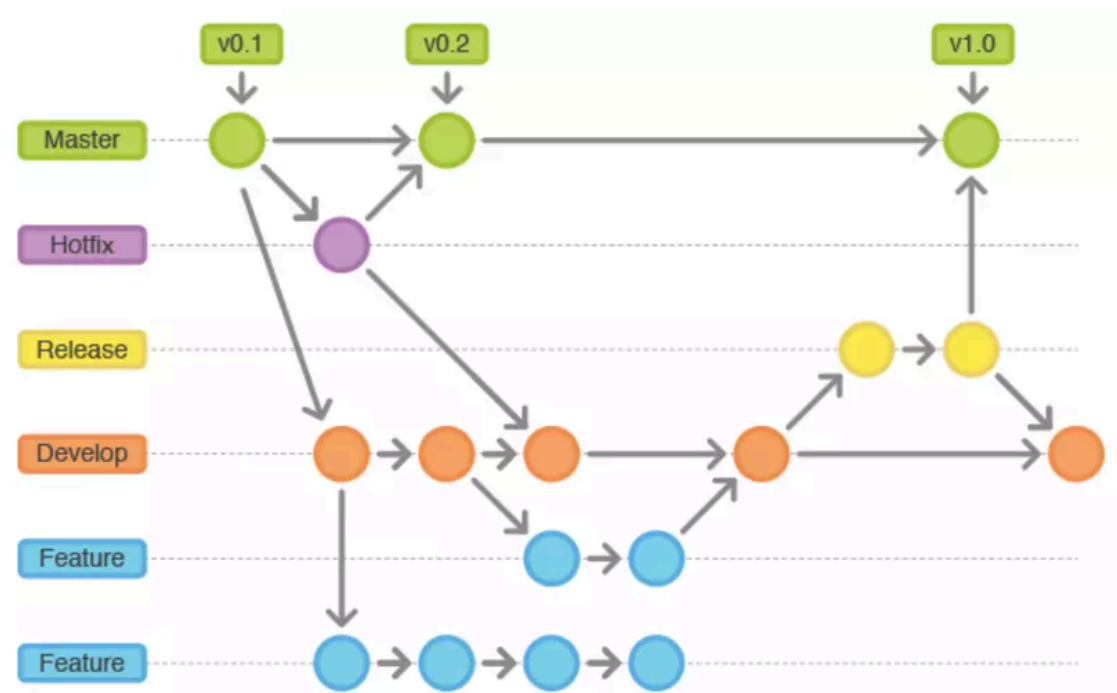


Buổi 4: Mô Hình Git Flow Trong Doanh Nghiệp

Phân loại chức năng từng nhánh trong dự án thực tế

Phân Loại Nhánh Chuẩn:

- **Main / Master:** Code chạy trực tiếp cho khách hàng (Production-ready).
- **Develop:** Nơi chứa code mới nhất đang thử nghiệm.
- **Feature Branches:** Nhánh làm tính năng riêng (VD: `feature/login`).
- **Hotfix Branches:** Nhánh vá lỗi gấp trực tiếp trên `main`.



Mini Practice 4.1

Cả nhóm thống nhất tạo nhánh `develop` từ `main`. Mỗi thành viên rẽ nhánh `feature/ten-minh` từ `develop` để làm bài tập, sau đó mở PR gộp lại vào `develop`.

Buổi 4: Các Extension Hỗ Trợ Git Trên VS Code

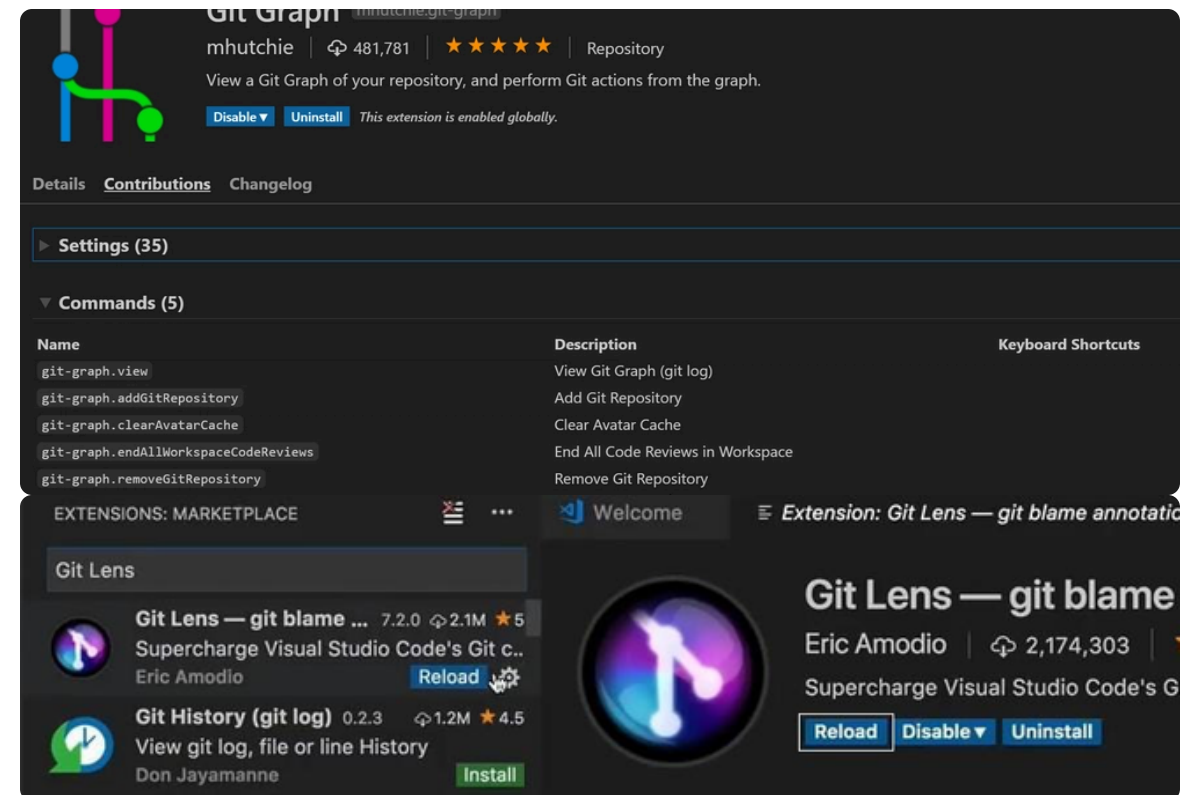
Tăng tốc độ làm việc và soi lịch sử code trực quan

1. GitLens

Hiển thị trực tiếp ai là người viết từng dòng code (Git Blame) và lịch sử chỉnh sửa theo thời gian thực.

2. Git Graph

Vẽ sơ đồ cây các nhánh, các điểm Commit & Merge bằng giao diện hình ảnh đẹp mắt, dễ thao tác.





BUỔI 5: OVERVIEW NỘI DUNG HỌC

Quản Lý Dự Án Jira & Liên Kết Tự Động Với GitHub

Các Nội Dung Chính

1. Giới thiệu các Tool quản lý công việc: Jira, Trello & GitHub Projects.
2. Khởi tạo Project & thiết lập Board (Kanban/Scrum) trên Jira.
3. Quản lý các loại Issue (Epic, Story, Task, Bug).
4. Tự động hóa liên kết Task ID Jira với Git Commit & Pull Request.



Buổi 5: Tạo Project & Quản Lý Task Trên Jira

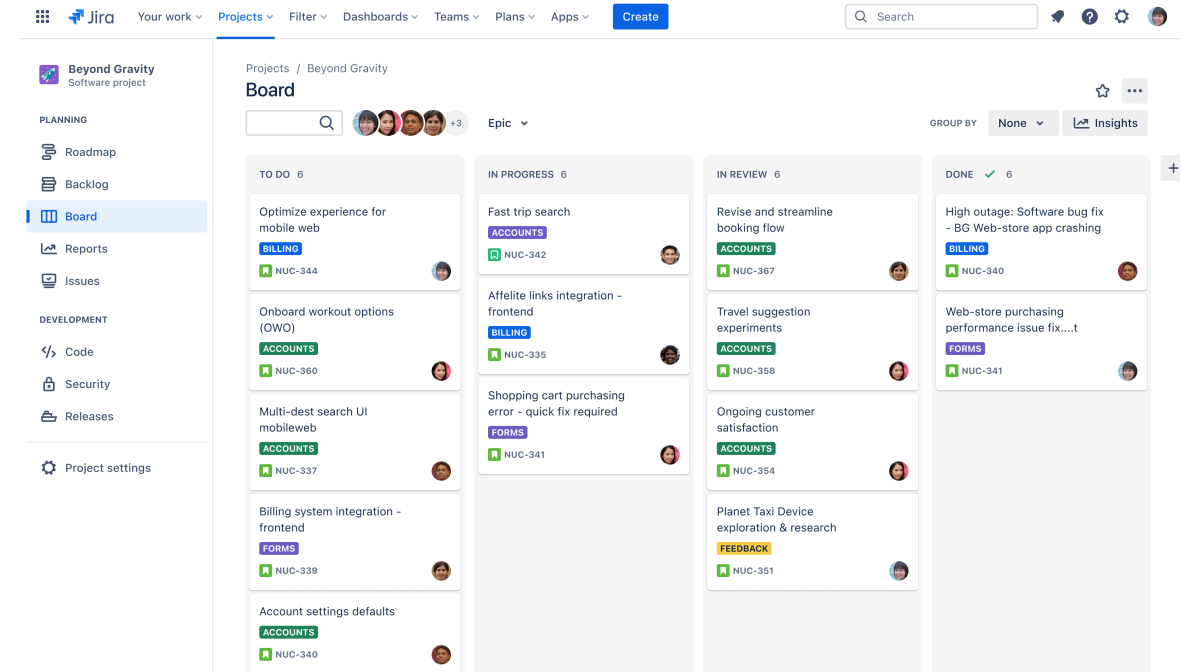
Theo dõi tiến độ dự án chuyên nghiệp

1. Các Cột Căn Bản Trên Board:

- To Do → In Progress → Code Review → Done.

2. Phân Loại Issue Trên Jira:

- Epic: Tính năng lớn dự án.
- Story / Task: Yêu cầu công việc cụ thể (có Mã ID: ECOMM-101).
- Bug: Lỗi phát sinh cần sửa.



Mini Practice 5.1

Đăng nhập Jira, tạo 1 Software Project mẫu, tạo 2 Task trong cột **To Do** và gán Assignee cho bản thân.



Buổi 5: Hướng Dẫn Tích Hợp Jira Với GitHub

Các bước thiết lập liên kết hai nền tảng

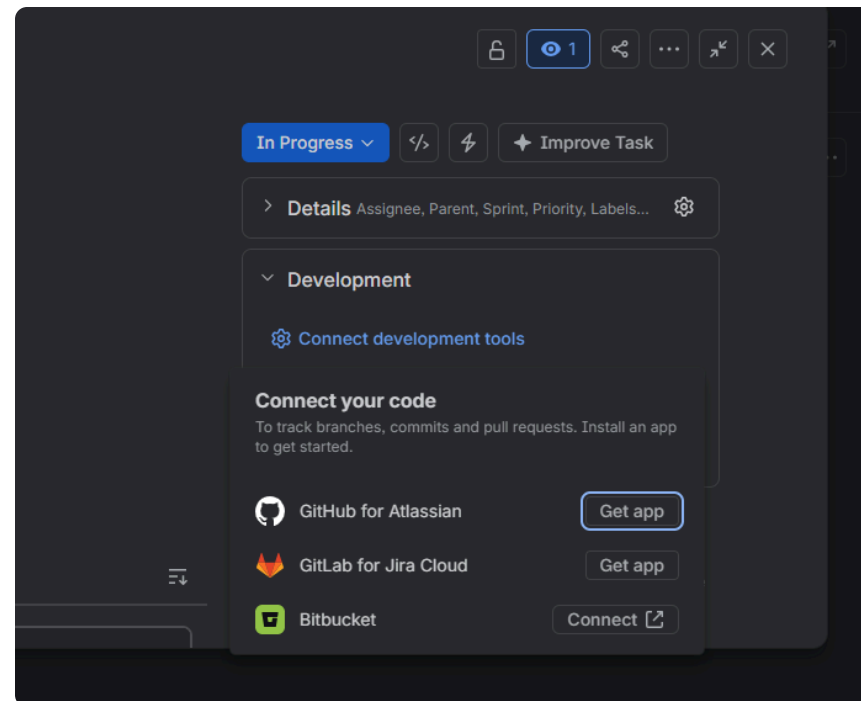
4 Bước Kết Nối Hệ Thống:

Bước 1: Vào Jira → Chọn **Apps** → Tìm app **GitHub for Jira** và bấm *Get it now*.

Bước 2: Đăng nhập tài khoản GitHub → Ủy quyền cấp phép (Authorize Atlassian).

Bước 3: Chọn Organization / Repository trên GitHub muốn kết nối với Jira Project.

Bước 4: Kiểm tra mục **Development** hiển thị trong giao diện chi tiết Task của Jira.



Buổi 5: Quy Trình Tự Động Hóa Jira + GitHub

Chỉ cần gắn Mã Task ID, Jira sẽ tự động cập nhật!

Luồng Tích Hợp Đã Tối Ưu Hóa

Giai Đoạn	Thao Tác Jira	Thao Tác Dưới Git / GitHub
1. Nhận Task	Lấy mã Task ID (VD: ECOMM-101)	<code>git checkout -b feature/ECOMM-101-login</code>
2. Viết Code	Chuyển Task sang In Progress	<code>git commit -m "ECOMM-101 Add validation"</code>
3. Code Review	Jira tự động gắn link PR vào Task	Push nhánh & Tạo PR chỉ định Reviewer
4. Hoàn Thành	Task tự động chuyển sang Done	Merge Pull Request vào nhánh chính

Buổi 5: Quy Trình Tự Động Hóa Jira + GitHub

Chỉ cần gắn Mã Task ID, Jira sẽ tự động cập nhật!

THING-1

Car

🔗 📄 🔗 ⬆ ⬆ ⚙️ ⋮

Description
Add a description...

Linked issues +

depends on

FEAT-2	Wheel	=	👤	BACKLOG
FEAT-4	Seat	=	👤	BACKLOG

is blocked by

FEAT-1	Engine	=	👤	BACKLOG
FEAT-3	Chassis	=	👤	BACKLOG

Your work Projects More + 🔍 Search 🔔 ? ⚙️ RW

Transition Linked Issues to Done NEW

Rule details **Turn on rule** Return to templates

⚙️ This rule was generated by Atlassian Intelligence
Review your rule and turn it on.
Try again
How was your experience? 🍌 🍎

When: Issue transitioned TO Done

BRANCH

For: Linked issues
Types: All link types

Then: Transition the issue to **DONE**

+ Add to branch

+ Add component

Destination status
DONE

Ensure a transition from the issue's source status to your selected destination status exists; [more info](#)

+ add regex to distinguish between multiple transitions to the same status

⚙️ Choose fields to set...

Resolution
Done



TỔNG KẾT KHÓA ĐÀO TẠO

Đây là phần câu hỏi

